

Domain Adaptation: SVHN to MNIST

Purpose

The purpose of this assignment is to become familiar with [DANN](#)¹, a popular unsupervised Domain Adaptation paper that uses the principle of adversarial learning to align the source and target datasets. You will implement Domain-Adversarial Training of Neural Networks on SVHN to MNIST datasets.

Objectives

Learners will be able to

- Gain practical experience implementing the DANN neural network
- Understand the architecture of models like FeatureExtractor, LabelClassifier, and DomainClassifier.

Technology Requirements

- GPU environment
- Jupyter Notebook
- Python3 (Python 3.8 and above)
- PyTorch
- Torchvision
- Numpy
- Matplotlib

¹Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., & Lempitsky, V. (2016). Domain-adversarial training of neural networks. *Journal of Machine Learning Research*, 17, 1–35.

Directions

Accessing ZyLabs

You will complete and submit your work through zyBooks's zyLabs. Follow the directions to correctly access the provided workspace:

1. Go to the Canvas Assignment, "**Submission: Domain Adaptation: SVHN to MNIST Assignment**"
2. Click the "**Load Submission...in new window**" button.
3. Once in ZyLabs, click the **green button** in the Jupyter Notebook to get started.
4. Review the directions and resources provided in the description.
5. When ready, review the provided code and develop your work where instructed.

Assignment Directions

In this assignment, we will treat the SVHN dataset of digits as the source domain and the MNIST dataset of digits as the target domain. Note that both domains need to have the same categories.

Design the Network Modules

Define the network architectures:

- Convolution Layers are represented as 'Conv(ch)', where 'ch' is the number of output channels. All convolutions are the same convolutions with filter size = 3, stride = 1, and padding = 1.
- Linear Layers are represented as 'Linear(fts)', where 'fts' is the number of output features.
- Every Convolution and Linear Layer is followed by ReLU activation except if it is the last layer before the Loss function.
- MaxPool layers are represented as 'MaxPool(fs)' where 'fs'=2 is the filter size with stride = 2.
- BatchNorm layers are represented as 'BatchNorm(ch)', where 'ch' is the number of channels or inputs feature dimensions. For Convolution layers use 'BatchNorm2d(ch)' and for Linear layers, use 'BatchNorm1d(ch)'.

FeatureExtractor Network is used to extract the features from the source and the target data.

FeatureExtractor (Input = Batch_Size x 3 x 32 x 32 or Batch_Size x 3 x 28 x 28):

Conv(32) → Conv(32) → MaxPool(2) → BatchNorm2d(32) → Conv(64) → Conv(64) → MaxPool(2) → BatchNorm2d(64) → Conv(128) → Conv(128) → AdaptiveAvgPool2d(1) → Linear(128) → BatchNorm1d(128)

LabelClassifier Network is used to classify the features (output of FeatureExtractor) into 10 categories

LabelClassifier (Input = Batch_Size x 128):

Linear(64) → BatchNorm1d(64) → Linear(10) → SoftmaxCrossEntropyLoss

DomainClassifier Network trains to distinguish between the source and target features. The input to the network is the features from FeatureExtractor.

DomainClassifier(Input = Batch_Size x 128)

Linear(64) → BatchNorm1d(64) → Linear(64) → BatchNorm1d(64) → Linear(1) → SigmoidBinaryCrossEntropyLoss

Initialize Network Module Objects

- You need to initialize the objects for the three network modules
- Initialize the optimizer and the scheduler for the learning rate
- Define a 'ce_criterion' using the nn.CrossEntropyLoss object to implement the softmax and the cross entropy loss

Classifier Logits

Define a function for forward pass through the FeatureExtractor and the LabelClassifier. The function propagates input x through the Feature extractor and the LabelClassifier.

Inputs:

x: Tensor of input images of dimensions **(m,3,s,s)**, where **m** is the number of samples, **3** is for the RGB channels and **s** is the image size.

Outputs:

logits: Tensor of outputs logits of dimensions **(m,10)**, where **m** is the number of samples, **10** is the number of categories.

Label Classifier Loss

Define a function that gives the classification loss for the LabelClassifier module. The input is a Tensor of images along with corresponding labels. The images are input to the FeatureExtractor followed by the LabelClassifier and the loss is calculated using CrossEntropy Use the 'calc_clf_logits()' function and the 'ce_criterion' nn.CrossEntropyLoss Object to calculate the loss.

The function returns the CrossEntropyLoss error for classifying `img` where ground truth is labels

Inputs:

img: Tensor of input images of dimensions **(m,3,s,s)**, where **m** is the number of samples, **3** is for the RGB channels and **s** is the image size.

Y: ground truth labels in Tensor of shape **(m)** with values in {0,1,,2, ..., 9}

Outputs:

clf_loss: scalar loss for the **m** images.

Evaluate Model

Define a function 'evaluate_model(.)' that returns the accuracy of classification for given input images `X` and ground truth labels `Y`.

The function returns the accuracy of classification

Inputs:

X: Tensor of input images of dimensions **(m,3,s,s)**, where **m** is the number of samples, **3** is for the RGB channels, **s** is the image size

Y: ground truth labels in Tensor of shape **(m)** with values in {0,1,,2, ..., 9}

Outputs:

acc: accuracy of classification

Train Classifier Model With Source Data

You will train just FeatureExtractor and LabelClassifier with the source data and evaluate classification accuracies on the target data. The trained model is called 'Source-only'. Ttrain the FeatureExtractor and the LabelClassifier modules using the source data.

Evaluate Classification Accuracies With Source-only Model

Calculate accuracy on source and target datasets. Notice how the Network does not perform well on the target data.

Reinitialize the Domain Alignment network

Reintalize the network with the same parameters for DANN training. This time we will use the DomainClassifier to align the domains in the DANN style

Importance of Domain Alignment: The DomainClassifier gradient can conflict with the training of the LabelClassifier with noisy gradients in the initial stages of the training. The parameter λ or 'lam' controls the importance of DomainClassifier in the overall objective of the domain alignment. Therefore, set $\lambda = 0$ initially and gradually increase it after the FeatureExtractor has stabilized.

$$\lambda = \frac{2}{1 + \exp(-\gamma.p)} - 1$$

where, $\gamma = 10$ and $p = \frac{itr + epoch \times no\ iters\ per\ epoch}{n\ epochs \times no\ iters\ per\ epoch}$ is the fraction of total iterations completed in the training.

Define a function that returns the scheduled value of lam based on epoch and iteration number.

Inputs:

itr: the iteration number in an epoch

epoch: the epoch number

no_itrs_per_epoch: the number of iterations (number of mini-batches) in an epoch

n_epochs: total number of epochs in the training

Domain Classification Loss

Similar to the Label Classifier Loss function, define a Domain Classifier Loss function which will return the loss of domain classification. The input is a Tensor of source images 's_img', a Tensor of target images 't_img', and the λ parameter 'lam'. You need to create the labels for the ground truth. The source images belong to class 1 and the target images to class 0. Propagate 's_img' through the FeatureExtractor and 't_img' through the FeatureExtractor. Concatenate the outputs and propagate the concatenated result through the DomainClassifier to obtain the logits. Determine the loss based on the logits and the ground truth.

It may not be possible to concatenate 's_img' and 't_img' and forward propagate the concatenated output through the FeatureExtractor and DomainClassifier - there may be a dimension mismatch.

Train and Evaluate the DANN Network

Train and calculate accuracy on source and target datasets using DANN. Observe the improvement in the target classification accuracies with the alignment of the source and target datasets. You should observe the accuracy of the target increase by at least 10 points compared to Source-only training. The settings mentioned here yielded a target test accuracy of 71%. Modify the network and the training procedure to the extent allowed to improve the target classification accuracies.

Submission Directions for Project Deliverables

Learners are expected to work on the assignment individually. Ideas and concepts may be discussed with peers or other sources can be referenced for assistance, but the submitted work must be entirely your own.

You must complete and submit your work through zyBooks's zyLabs to receive credit for the assignment:

1. To get started, use the provided Jupyter Notebook in your workspace.
2. All necessary datasets are already loaded into the workspace.
3. Execute your code by clicking the “**Run**” button in top menu bar.
4. When you are ready to submit your completed work, click on “**Submit for grading**” located on the bottom left from the notebook.
5. You will know you have completed the assignment when feedback appears below the notebook.
6. If needed: to resubmit the assignment in zyLabs
 - a. Edit your work in the provided workspace.
 - b. Run your code again.
 - c. Click “**Submit for grading**” again at the bottom of the screen.

Your submission score will automatically be populated from zyBooks into your course grade. However, the course team will review submissions after the due date has passed to ensure grades are accurate.

Evaluation

This assignment is auto-graded. There are a total of ten (10) test cases and each has points assigned to it. Please review the notebook to see the points assigned for each test case. A percentage score will be passed to Canvas based on your score.